

EXPERIMENT 28

By Trishil Chauhan

Aim

To study variable-length two-dimensional arrays in C, column-major input using nested loops, row-major traversal using a linear index, and copying a 2D array into a 1D array in row-major order.

Question 1 - Variable-Length 2D Array

Algorithm

1. Start.
2. Read rows and columns. Repeat input until both values are positive.
3. Declare a variable-length 2D integer array using the accepted dimensions.
4. Read elements column by column: outer loop over columns and inner loop over rows.
5. Print elements in row-major order using one loop. For linear index k , the corresponding element is $a[k / \text{cols}][k \% \text{cols}]$.
6. Stop.

C Program

```
#include <stdio.h>

int main(void) {
    int rows, cols, i, j, k;

    do {
        printf("Enter number of rows and columns: ");
        scanf("%d %d", &rows, &cols);
        if (rows <= 0 || cols <= 0)
            printf("Both values must be positive. Try again.\n");
    } while (rows <= 0 || cols <= 0);

    int a[rows][cols];

    printf("Enter array elements column by column:\n");
    for (j = 0; j < cols; j++) {
        for (i = 0; i < rows; i++) {
            printf("a[%d][%d] = ", i, j);
            scanf("%d", &a[i][j]);
        }
    }

    printf("Elements in row-major order:\n");
    for (k = 0; k < rows * cols; k++)
        printf("%d ", a[k / cols][k % cols]);

    printf("\n");
    return 0;
}
```

Example

For rows = 2 and columns = 3, if column-wise input is 1, 2, 3, 4, 5, 6, the array becomes:

1 3 5

2 4 6

Row-major output: 1 3 5 2 4 6

Question 2 - Copy 2D Array into 1D Array

Algorithm

1. Start and read positive values of rows and columns.
2. Declare a variable-length 2D array and a 1D array of size rows * columns.
3. Read the 2D array elements in normal row-major order.
4. Use nested loops to copy each 2D element into the next position of the 1D array.

5. Display the original 2D array.
6. Display the copied 1D array.
7. Stop.

C Program

```
#include <stdio.h>

int main(void) {
    int rows, cols, i, j, k = 0;

    do {
        printf("Enter number of rows and columns: ");
        scanf("%d %d", &rows, &cols);
        if (rows <= 0 || cols <= 0)
            printf("Both values must be positive. Try again.\n");
    } while (rows <= 0 || cols <= 0);

    int a[rows][cols];
    int oneD[rows * cols];

    printf("Enter elements of the 2D array:\n");
    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            scanf("%d", &a[i][j]);

    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            oneD[k++] = a[i][j];

    printf("2D array:\n");
    for (i = 0; i < rows; i++) {
        for (j = 0; j < cols; j++)
            printf("%d ", a[i][j]);
        printf("\n");
    }

    printf("1D array in row-major order:\n");
    for (i = 0; i < rows * cols; i++)
        printf("%d ", oneD[i]);

    printf("\n");
    return 0;
}
```

Example

For a 2 x 3 array containing:

1 2 3

4 5 6

the copied 1D array is: 1 2 3 4 5 6

Test-Case Table

Question	Input / condition	Expected result
1	rows=2, cols=3; column input: 1 2 3 4 5 6	Row-major output: 1 3 5 2 4 6
1	rows=0, cols=3	Reject dimensions and ask again
2	2 x 3 array: 1 2 3 / 4 5 6	1D array: 1 2 3 4 5 6
2	rows=-2, cols=4	Reject dimensions and ask again

Result

Both programs were implemented successfully using variable-length arrays. The first program accepts elements column by column and displays them in row-major order through a single linear index. The second program copies a 2D array into a 1D array in row-major order and displays both arrays.